

Document Number: P0574r1
Date: 2017-03-02
Author: [Anthony Williams](#)
Just Software Solutions Ltd
Reply-To: [Anthony Williams](#)
Just Software Solutions Ltd
[Bryce Adelstein Lelbach](#)
Lawrence Berkeley National Laboratory
Audience: SG1, LWG

P0574r1: Algorithm Complexity Constraints and Parallel Overloads

This paper addresses NB comments CH 10.

Introduction

The C++17 draft features overloads of a lot of the standard library algorithms which take an `ExecutionPolicy` argument, in order to allow parallel implementations. However, the requirements of many of the algorithms are written in such a way as to require a sequential implementation, or a sub-optimal parallel implementation, thus eliminating some or all of the benefit in the new overloads.

[P0523r1](#) attempts to address this problem by a blanket relaxation of the constraints for overloads with an `ExecutionPolicy`. I think this is an incorrect solution. It simultaneously grants too much leeway for some algorithms, while not fixing the problems with others. Instead, I think it is better to address each algorithm individually.

In this paper I have gathered those algorithms where the requirements are specified in such a way as to rule out an efficient parallel implementation, and proposed alternative specifications for those algorithms.

This paper has been updated since [P0574r0](#) to incorporate changes suggested by SG1 at Kona 2017. Notably:

- `std::execution::sequential_policy` is no longer special; it is treated as any other `ExecutionPolicy` from the point of view of complexity constraints.
- The new constraints on the iterators and value types have now been changed to notes about performance consequences rather than normative wording.
- Section numbers in the proposed wording now refers to numbers from the latest C++17 draft.
- Added constraints on `exclusive_scan`, `inclusive_scan`, `transform_exclusive_scan` and `transform_inclusive_scan`.

It has also been updated to incorporate changes suggested by LWG at Kona 2017, notably:

- Rebase against the latest working paper and [P0467r2](#).
- Add blanket wording to **[numeric.ops.overview]** stating that `BinaryOperation`, etc are function objects.
- Applied a fix from [P0574r1](#) which corrects the new numeric algorithms requirements to forbid invalidation of the last iterator.
- Move `unique_copy`, `unique_copy_if` and `partition_copy` to [P0467r2](#).
- Fix the parallel specification of `adjacent_difference` and move it to [P0467r2](#).

Algorithms and proposed wording

The following changes are relative to the latest working paper as of 03-02-2017 with the changes from [P0467r2](#) and [P0452r1](#) applied.

25.5.8 Adjacent find [alg.adjacent.find]

Modify the complexity for `adjacent_find` paragraph 2 as follows:

2 Complexity: **For the overloads with no `ExecutionPolicy`** ~~For a nonempty range~~, exactly $\min((i - \text{first}) + 1, (\text{last} - \text{first}) - 1)$ applications of the corresponding predicate, where i is `adjacent_find`'s return value. **For the overloads with an `ExecutionPolicy`, $O(\text{last} - \text{first})$ applications of the corresponding predicate.**

25.6.1 Copy [alg.copy]

Modify the requirements for `copy_if` in paragraph 12 as follows:

12 *Requires:* The ranges `[first, last)` and `[result, result + (last - first))` shall not overlap. [Note: For the overloads with an `ExecutionPolicy`, there may be a performance cost if `iterator_traits<ForwardIterator1>::value_type` is not `MoveConstructible`. — end note]

25.6.8 Remove [alg.remove]

Modify the requirements for `remove_copy` and `remove_copy_if` in paragraph 7 as follows:

7 *Requires:* The ranges `[first, last)` and `[result, result + (last - first))` shall not overlap. The expression `*result = *first` shall be valid. [Note: For the overloads with an `ExecutionPolicy`, there may be a performance cost if `iterator_traits<ForwardIterator1>::value_type` is not `MoveConstructible`. — end note]

25.7.2 Nth element [alg.nth.element]

Modify the complexity for `nth_element` in paragraph 3 as follows:

3 *Complexity:* For the overloads with no `ExecutionPolicy`, linear on average. For the overloads with an `ExecutionPolicy`, $O(N)$ applications of the predicate, and $O(N \log N)$ swaps, where $N = \text{last} - \text{first}$.

25.7.4 Partitions [alg.partitions]

Modify the complexity for `partition` in paragraph 7 as follows:

7 *Complexity:* If `ForwardIterator` meets the requirements for a `BidirectionalIterator`, at most $(\text{last} - \text{first}) / 2$ swaps are done; otherwise at most $\text{last} - \text{first}$ swaps are done. Exactly $\text{last} - \text{first}$ applications of the predicate are done. Let $N = \text{last} - \text{first}$:

- For the overloads with no `ExecutionPolicy`, exactly N applications of the predicate. At most $N / 2$ swaps if `ForwardIterator` meets the `BidirectionalIterator` requirements and at most N swaps otherwise.
- For the overloads with an `ExecutionPolicy`, $O(N \log N)$ swaps and $O(N)$ applications of the predicate.

Modify the complexity for `stable_partition` in paragraph 11 as follows:

11 *Complexity:* At most $N \log(N)$ swaps, where $N = \text{last} - \text{first}$, but only $O(N)$ swaps if there is enough extra memory. Exactly $\text{last} - \text{first}$ applications of the predicate. Let $N = \text{last} - \text{first}$:

- For the overloads with no `ExecutionPolicy`, at most $N \log N$ swaps, but only $O(N)$ swaps if there is enough extra memory. Exactly N applications of the predicate.
- For the overloads with an `ExecutionPolicy`, $O(N \log N)$ swaps and $O(N)$ applications of the predicate.

25.7.5 Merge [alg.merge]

Modify the complexity for `merge` in paragraph 4 as follows:

4 *Complexity:* For the overloads with no `ExecutionPolicy`, at most $(\text{last1} - \text{first1}) + (\text{last2} - \text{first2}) - 1$ comparisons. For the overloads with an `ExecutionPolicy`, $O((\text{last1} - \text{first1}) + (\text{last2} - \text{first2}))$ comparisons.

Modify the complexity for `inplace_merge` in paragraph 8 as follows:

8 *Complexity:* When enough additional memory is available, $(\text{last} - \text{first}) - 1$ comparisons. If no additional memory is available, an algorithm with complexity $N \log(N)$ may be used, where $N = \text{last} - \text{first}$. Let $N = \text{last} - \text{first}$:

- For the overloads with no `ExecutionPolicy`, if enough additional memory is available, exactly $N - 1$ comparisons.
- For the overloads with no `ExecutionPolicy` if no additional memory is available, and for the overloads with an `ExecutionPolicy`, $O(N \log N)$ comparisons.

26.8.1 Header <numeric> synopsis [numeric.ops.overview]

Add the following paragraph after paragraph 1:

1 The requirements on the types of algorithms' arguments that are described in the introduction to Clause 25 also apply to the following algorithms.

2 Throughout this Clause, the UnaryOperation, BinaryOperation, BinaryOperation1 and BinaryOperation2 parameters are used whenever an algorithm expects a function object (20.14).

26.8.3 Reduce [reduce]

Modify the requirements for `reduce` in paragraph 5 as follows:

5 *Requires:*

- `T` shall be `MoveConstructible` (Table 23).
- All of `binary_op(init, *first)`, `binary_op(*first, init)`, `binary_op(init, init)`, and `binary_op(*first, *first)` shall be convertible to `T`.
- `binary_op` shall neither invalidate iterators or subranges, nor modify elements in the range `[first, last)`.

26.8.4 Transform reduce [transform.reduce]

Modify the requirements for `transform_reduce` in paragraph 1 as follows:

1 *Requires:*

- `T` shall be `MoveConstructible` (Table 23).
- All of `binary_op(init, unary_op(*first))`, `binary_op(unary_op(*first), init)`, `binary_op(init, init)`, and `binary_op(unary_op(*first), unary_op(*first))` shall be convertible to `T`.
- Neither `unary_op` nor `binary_op` shall invalidate subranges, or modify elements in the range `[first, last)`.

26.8.7 Exclusive scan [exclusive.scan]

Modify the requirements for `exclusive_scan` in paragraph 3 as follows:

3 *Requires:*

- `T` shall be `MoveConstructible` (Table 23).
- All of `binary_op(init, *first)`, `binary_op(init, init)`, and `binary_op(*first, *first)` shall be convertible to `T`.
- `binary_op` shall neither invalidate iterators or subranges, nor modify elements in the ranges `[first, last)` or `[result, result + (last - first))`.

26.8.8 Inclusive scan [inclusive.scan]

Modify the requirements for `inclusive_scan` in paragraph 3 as follows:

3 *Requires:*

- If `init` is provided, `T` shall be `MoveConstructible` (Table 23); otherwise, `ForwardIterator1`'s value type shall be `MoveConstructible` (Table 23).
- If `init` is provided, all of `binary_op(init, *first)`, `binary_op(init, init)`, and `binary_op(*first, *first)` shall be convertible to `T`; otherwise, `binary_op(*first, *first)` shall be convertible to `ForwardIterator1`'s value type.
- `binary_op` shall neither invalidate iterators or subranges, nor modify elements in the ranges `[first, last)` or `[result, result + (last - first))`.

26.8.9 Transform exclusive scan [transform.exclusive.scan]

Modify the requirements for `transform_exclusive_scan` in paragraph 1 as follows:

1 *Requires:*

- `T` shall be `MoveConstructible` (Table 23).
- All of `binary_op(init, unary_op(*first))`, `binary_op(init, init)`, and `binary_op(unary_op(*first), unary_op(*first))` shall be convertible to `T`.
- Neither `unary_op` nor `binary_op` shall invalidate subranges, or modify elements in the ranges `[first, last)` or `[result, result + (last - first))`.

26.8.10 Transform inclusive scan [transform.inclusive.scan]

Modify the requirements for `transform_inclusive_scan` in paragraph 1 as follows:

1 *Requires:*

- If `init` is provided, `T` shall be `MoveConstructible` (Table 23); otherwise, `ForwardIterator1`'s value type shall be `MoveConstructible` (Table 23).
- If `init` is provided, all of `binary_op(init, unary_op(*first))`, `binary_op(init, init)`, and `binary_op(unary_op(*first), unary_op(*first))` shall be convertible to `T`; otherwise, `binary_op(unary_op(*first), unary_op(*first))` shall be convertible to `ForwardIterator1`'s value type.
- Neither `unary_op` nor `binary_op` shall invalidate iterators or subranges, or modify elements in the ranges `[first, last)` or `[result, result + (last - first))`.

Direction to the Editor: Add a footnote to each use of fully-closed ranges in the changes above that apply to 26.8 [numeric.ops] that says "The use of fully closed ranges is intentional".